

Legal Consistency & Change Intelligence

AI Reasoning Contract — Production Specification v1.2

Supersedes v1.1. Correction pass only — no new features, no architecture changes beyond the eight items below. Everything not listed here is unchanged from v1.1; read this as a diff.

Change 1 — `coexistence_possible`: unambiguous null rule

Final rule, stated once, applied everywhere: `coexistence_possible` is populated (`true` | `false` | `"uncertain"`) **if and only if** `relationship == "apparent_conflict"`. For every other relationship value, it is `null` — not omitted, not optional, always explicitly `null`.

Validator addition (new step, runs alongside the existing evidence-count/coexistence step): regardless of what the model outputs, the server forces `coexistence_possible = null` on any finding where `relationship != "apparent_conflict"`. This is defensive overwriting, not trust — the field is never read as meaningful on non-conflict findings even if the model populates it.

Change 2 — `removal_evidence_completeness`: fully server-derived, never model-requested

This field is **removed from the model's output schema entirely**. The model is never asked to produce it and any value it might emit there is ignored.

`removal_evidence_completeness` is computed exclusively by the server, from the deterministic full-document counterpart search described in v1.1 §5:

- If the cited clause of a `removal` finding carries `orphan_confirmed` status (set by the server-side full-document search, independent of the model), the server sets `removal_evidence_completeness = "full_document_search"` on that finding.
- If the cited clause does **not** carry `orphan_confirmed` status — meaning the server-side search either wasn't run, found a counterpart, or the finding cites a clause outside the confirmed-orphan set — the server downgrades the finding's `relationship` to `"uncertain"` and does not set `removal_evidence_completeness` at all (it's only meaningful on `removal` findings).

The model's role in a `removal` finding is limited to: identifying that a clause appears to have no counterpart, citing that clause, and describing what's missing. Whether that claim is *actually* verified against the full document set is entirely a server determination, never a model assertion.

Change 3 — Exactly two documents for the production MVP

API contract for Attempt 1: the analysis endpoint accepts **exactly two documents**, no more, no fewer. This is enforced as a deterministic pre-call validation step — a request with 1 or 3+ documents is rejected with a clear error before any extraction or model call occurs.

Domain model: stays array-based internally (`documents: Document[]`), and the output schema's `evidence: CitationRef[]` still supports citations spanning more than two documents structurally — this costs nothing and is what makes future extension possible without a schema migration. But this is now explicitly **unused, untested, unshipped capacity** for Attempt 1, not a soft "2, occasionally more" allowance. The system prompt is rewritten to say "two documents," not "two, occasionally more" (v1.1's system prompt wording, now corrected).

Change 4 — Hedge/label consistency check: scoped correctly

Problem with v1.1: the check as written could fire on hedge language anywhere in a finding, including `potential_consideration`, `uncertainty`, and `suggested_question` — fields that are *supposed* to sound exploratory/soft by design (`uncertainty` in particular will almost always contain words like "unclear" or "not established"). Applying the check there would incorrectly downgrade findings whose actual relationship classification is perfectly sound.

Corrected rule: the hedge/label consistency check evaluates **only** `description` and `ai_interpretation` — the two fields that state what the relationship actually *is*. If either of those two fields contains hedge language ("may," "possibly," "unclear," "not certain") while `relationship` is a confident non-`uncertain` label, that's a real inconsistency (the model is describing uncertainty while asserting confidence) and the finding is downgraded. Hedge language in `potential_consideration`, `uncertainty`, or `suggested_question` triggers nothing — it's expected and correct in those fields.

Change 5 — No separate `deadlines[]` field, plus fix to `obligations[].deadline` inconsistency

Confirming existing behavior: deadline information is represented exclusively inside `obligations[].deadline`. There is no standalone `deadlines[]` array anywhere in the schema, and none should be added — a deadline with no associated obligation text and citation isn't a valid output shape in this contract.

Two gaps this left unaddressed, now fixed:

1. **Format of `deadline` was undefined.** `deadline` must hold the deadline **as grounded in the cited clause's text** — a relative description ("30 days from invoice receipt," "prior to the renewal date") or, only if the clause itself states one, a fixed calendar date. The model must never compute or resolve a relative deadline into an absolute date on its own (e.g., inferring an actual calendar date from "30 days before the anniversary")

without the anniversary date being stated) — that's an unsupported inference, not an extraction, and is exactly the kind of silent hallucination risk this contract exists to prevent. If the clause references a date/anchor not itself stated in the supplied documents, the correct output is an `unresolved_references` entry and/or an `uncertain` finding, not a computed `deadline` value.

2. **Validation coverage for `obligations[]` was implicit, not stated — and the first pass at stating it overcorrected.** Citation existence (pipeline step 4) applies identically to `obligations[].evidence` as to `findings[].evidence` — an obligation citing a nonexistent clause ID is dropped the same way an invalid finding is. But the prescriptive-language filter must **not** reject ordinary contractual normative language ("shall," "must," "is required to") appearing in `obligations[].text` or `description` when it reports what a named party is obligated to do per the cited clause — that is faithful extraction, exactly what these fields exist for. See the corrected filter rule immediately below.

Correction (this pass) — deadline key presence:

- **Key presence:** `deadline` is a required key on every `obligations[]` entry, always present, typed `string | null` — never an optional/absent key. An obligation with no stated deadline sets `deadline: null`; it never omits the key.
- **Grounding:** `deadline` must be the time period or date as it appears in, or is directly and unambiguously stated by, the evidenced clause text itself (e.g., "within 30 days of invoice receipt," or an explicit calendar date the clause states outright). The model must never calculate or resolve an absolute date from a relative period (e.g., turning "30 days after execution" into a specific calendar date) unless the anchor date needed for that calculation is itself present in the cited evidence. If resolution would require information not in the evidenced text, `deadline` reports the relative period as stated, unresolved — it is never computed.

Correction (this pass) — prescriptive-language filter vs. legitimate obligation language:

The filter's actual test is **grammatical addressee, not vocabulary**. "Shall," "must," and "is required to" are exactly the words real contracts use, and `obligations[].text`/`description` exist to report them faithfully. The filter rejects a specific pattern — the assistant speaking to the user in the second person as a directive — not those words in general.

- **Allowed (document-reporting, third-person, evidence-grounded):** "The Contractor shall maintain insurance." / "The Vendor is required to deliver within 14 days." / "Payment must be made within 30 days of invoice receipt." — these describe what the document states a named party must do, cited to a clause. Reject nothing here.
- **Rejected (assistant-directed, second-person, advice-shaped):** "You must maintain insurance." / "You should sign this." / "You are required to accept these terms." — the assistant addressing the user directly with an imperative, or any of the existing

banned legal-conclusion assertions ("this is legal/illegal/enforceable/unenforceable/valid/void").

This distinction is the filter's actual rule everywhere it runs — `description`, `ai_interpretation`, `potential_consideration`, `suggested_question`, and `obligations[ ].text` all share this single test, not five separate rule sets. Pipeline step 8 (Consolidated Validation Pipeline, below) is corrected to state this explicitly.

Change 6 — `unrelated` vs. `uncertain`: raise the bar, add the adversarial test

Corrected instruction (system prompt): `unrelated` is reserved for cases where the documents **clearly** lack any meaningful legal relationship — different subject matter, different parties, no shared agreement context, nothing connecting them. Any case that isn't clear-cut defaults to `uncertain`, not `unrelated`. This matters because `unrelated` is the one status that **deletes all findings** (v1.1 §6 step 2) — a wrongly-confident `unrelated` call silently destroys real analysis, while a wrongly-cautious `uncertain` call merely caps finding specificity (v1.1 §6 step 8) without discarding anything. The asymmetric cost of being wrong is why the bar for `unrelated` is higher.

New adversarial test (#40): two documents that are substantively the same underlying agreement but share **low lexical overlap** — e.g., one drafted in dense defined-term legal prose, the other a plain-language restatement or a version with completely renamed parties/roles and reorganized section numbering. Expected: `document_relationship_assessment.status = "related"` (or, at worst, `"uncertain"` — never `"unrelated"`), because the substantive connection is real even though a naive keyword-overlap signal would look weak. This test exists specifically because the system has **no deterministic lexical pre-check gating document-relatedness** (that was deliberately not built — v1.0 §12 item 6) — the semantic judgment is entirely the model's, and this test verifies it isn't quietly failing on low-overlap-but-related pairs.

Change 7 — Efficient deterministic counterpart search (not $O(N^2)$)

Problem with v1.1 as written: "run deterministic counterpart-matching across the full clause sets of both documents" was underspecified and reads as an implicit all-pairs comparison, which is unbounded work as document/clause counts grow.

Corrected approach — inverted-index candidate matching, not brute-force all-pairs:

1. At clause-segmentation time, build a lightweight index per document: each clause is keyed by (a) its normalized `section_reference` if present, (b) a set of extracted defined/capitalized terms, and (c) a small token-shingle set (e.g., normalized 3-gram or keyword-set) for lexical overlap scoring.
2. For each clause in the original document, look up **candidate counterparts** in the revised document's index — only clauses sharing at least one index key (matching

section reference, a shared defined term, or shingle overlap above a low threshold) are considered candidates.

3. Full comparison (the actual counterpart-confirmation logic) runs only against that candidate set, not against every clause in the other document.
4. A clause is `orphan_confirmed` only if it has **zero** candidates in the index lookup, or every candidate is confirmed a non-match by the comparison logic.

This is still 100% deterministic, still no ML/embeddings (consistent with the standing "no vector infrastructure" decision), and bounds the work to roughly linear in clause count for realistic documents instead of quadratic. It is the same conceptual technique as an inverted index — cheap to build, cheap to query, and appropriate for the document sizes this product actually handles.

Change 8 — Three additional few-shot examples (system prompt only)

Added to the system prompt, not replacing the five existing examples: (a) a valid `removal` example demonstrating the model's limited role now that `removal_evidence_completeness` is server-only, (b) a semantic-equivalence example with deliberately low lexical overlap (different party names, different vocabulary, same substance), and (c) a conditional-modification example that must NOT be classified `apparent_conflict` because the text itself reconciles the apparent numeric discrepancy with a stated condition. Full text in the system prompt document.

Additional Adversarial Tests (v1.2)

40. (Change 6, already specified above) Low-lexical-overlap-but-related documents → `status` must be `related` or, at worst, `uncertain` — never `unrelated`.
41. **Exactly-two-document enforcement.** Submit a request with 1 document and, separately, a request with 3 documents → both must be rejected by the document-count gate (pipeline step 1) with a clean deterministic error, before any extraction or model call occurs (zero model calls in both cases).
42. **Server-enforced `coexistence_possible` nulling.** Using a fixture (not a live model call) where the raw model output sets `coexistence_possible` to `true` on a finding labeled `modification` (i.e., the model got Change 1's rule wrong) → the validator (pipeline step 6) must overwrite it to `null` in the rendered result, regardless of what the model emitted. This test exercises the validator directly against a deliberately non-conformant fixture, not the model's typical behavior.
43. **Prescriptive-language filter — document-reporting vs. assistant-directed.** Three fixtures against a source clause reading "The Contractor shall maintain commercial general liability insurance...": (a) an `obligations[.text]` value of "The Contractor shall maintain insurance." (document-reporting, third-person) must survive validation unchanged; (b) an equivalent document-derived obligation phrased with "must" or "is required to" in the same third-person, document-grounded register must

also survive; (c) an `obligations[].text` value of "You must maintain insurance." (assistant-directed, second-person) must be rejected by pipeline step 8. This directly tests the addressee-based distinction, not a vocabulary blocklist.

Everything Else

Unchanged from v1.1: domain model core, input/context/output schema shapes (aside from Changes 1/2/5 above), cache strategy, injection defense, advice-refusal classifier and patterns, document-uncertain blanket downgrade, dedup, disclaimer injection, testing architecture, implementation-security requirements for Antigravity, and the N-document extensibility posture (schema-ready, unshipped).

Consolidated Validation Pipeline (v1.2, full order)

1. **Document-count gate (new)** — reject any request with `len(documents) != 2` before extraction or any model call.
2. Schema validation.
3. Document-relationship gate — `unrelated` → drop `findings[]`; `uncertain` → flag for step 9 downgrade; `related` → proceed.
4. Citation existence check.
5. Evidence-count-per-relationship + coexistence relabel (apparent_conflict logic).
6. `coexistence_possible` **null-enforcement (new)** — force `null` on any finding where `relationship != apparent_conflict`.
7. Removal-evidence-completeness — **fully server-derived per Change 2**, never read from the model.
8. **Prescriptive-language filter — corrected rule (this pass)**. Rejects assistant-directed, second-person imperatives ("you must," "you should," "you need to," "you are required to," "you have the right to") and unsupported legal-conclusion assertions ("this is legal/illegal/enforceable/unenforceable/valid/void") in `description`, `ai_interpretation`, `potential_consideration`, `suggested_question`, and `obligations[].text`. Does **not** reject third-person, document-grounded normative language ("shall," "must," "is required to") reporting what a named party is obligated to do per a cited clause — see Change 5 for the allowed/rejected examples.
9. **Hedge/label consistency check — scoped to `description` + `ai_interpretation` only (corrected, Change 4)**.
10. Document-uncertain blanket downgrade.
11. Dedup.
12. Disclaimer + `injected_content_flags` injection.